Guide complet du projet Identity Service

Documentation fonctionnelle et technique

Architecture · API · sécurité · persistance · exploitation

Sommaire

1. Objet et périmètre . . . 3

2. Vue d'ensemble de l'architecture . . . 3

Dossiers . . . 4

3. Parcours fonctionnels . . . 4

3.1 Inscription . . . 4

3.2 Connexion et jeton . . . 4

3.3 Requête authentifiée . . . 4

3.4 Dépôt et examen KYC . . . 5

4. API HTTP . . . 5

5. Explication des classes Java . . . 5

5.1 Démarrage et configuration de sécurité . . . 5

5.2 Sécurité par jeton . . . 6

5.3 Contrôleurs REST . . . 7

5.4 Services métier . . . 7

5.5 Persistance : repositories et entités . . . 8

5.6 DTO et contrats de données . . . 9

5.7 Exceptions et format d'erreur . . . 9

6. Base de données et migrations . . . 9

7. Configuration et lancement . . . 10

application.yaml . . . 10

Docker . . . 10

Maven (pom.xml) . . . 10

8. Tests . . . 11

9. Points d'attention constatés dans le code . . . 11

10. Glossaire . . . 12

11. Résumé . . . 12

Documentation fonctionnelle et technique — version construite à partir du code du dépôt.

1. Objet et périmètre

`identity-service` est une API REST Java destinée à la gestion de l'identité et du processus KYC d'une application bancaire. Elle permet de créer un compte, de s'authentifier, de récupérer son profil, de déposer des pièces justificatives et de les faire examiner par un administrateur. Elle expose aussi une clé publique RSA au format JWKS pour permettre à d'autres services de vérifier les jetons signés par l'application.

Le service ne contient ni interface utilisateur ni logique de comptes bancaires ou de paiements. Les répertoires `static` et `templates` sont présents mais ne contiennent pas de ressources applicatives dans l'état examiné. Le périmètre réel est l'API backend et son stockage.

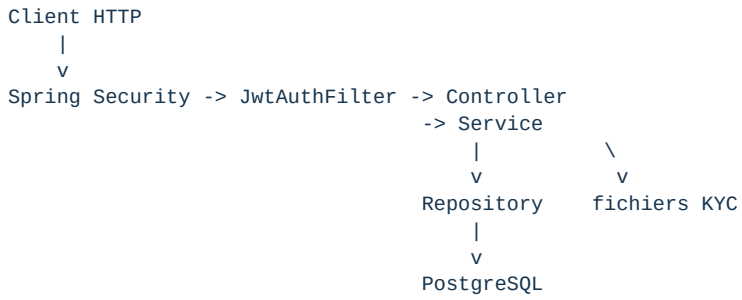
Technologies : Java 21, Spring Boot 3.4.2, Spring MVC, Spring Security, JPA/Hibernate, PostgreSQL, Flyway, JJWT, Lombok, Maven et Docker Compose.

2. Vue d'ensemble de l'architecture

Une requête suit généralement cette chaîne :

1. Le client appelle une route HTTP sur le port 8081.
2. Spring Security applique les règles d'accès. Pour les routes protégées, `JwtAuthFilter` lit le jeton Bearer, valide sa signature/expiration et charge l'utilisateur.
3. Le contrôleur traduit la requête HTTP en appel de service et choisit le code de réponse.
4. Le service porte les règles métier et délimite les transactions.
5. Les interfaces `repository` utilisent Spring Data JPA pour lire/écrire PostgreSQL.
6. Les entités représentent les tables; les DTO définissent les données échangées avec les clients.
7. `GlobalExceptionHandler` convertit certaines exceptions en réponses JSON cohérentes.

Pourquoi séparer ces couches ? Le contrôleur reste focalisé sur HTTP, le service sur le métier, le repository sur l'accès aux données et les DTO sur le contrat public. Cette séparation rend le code plus facile à faire évoluer et à tester sans exposer directement les entités de base de données.



Dossiers

Dossier	Responsabilité	Intérêt
`config`	Configuration de composants applicatifs, notamment les clés RSA	Centraliser les objets partagés et leur cycle de vie
`controller`	Routes REST et codes HTTP	Définir clairement l'API consommée par les clients
`dto`	Objets d'entrée et de sortie HTTP	Valider les entrées et maîtriser les champs exposés
`exception`	Exceptions métier, format d'erreur, traduction HTTP	Fournir des erreurs compréhensibles et uniformes
`model`	Entités JPA et énumérations métier	Représenter les données persistées
`repository`	Interfaces d'accès aux données	Réutiliser les opérations Spring Data sans SQL répétitif
`security`	Filtre JWT et génération/validation des jetons	Authentifier les requêtes sans session serveur
`service`	Cas d'utilisation et règles métier	Garder les décisions métier hors des contrôleurs
`src/main/resources/db/migration`	Scripts Flyway versionnés	Faire évoluer le schéma de manière répétable
`src/test`	Tests automatisés	Vérifier les comportements à mesure que le projet évolue

3. Parcours fonctionnels

3.1 Inscription

`POST /users/register` reçoit `RegisterRequest`. `@Valid` applique les contraintes de format, de présence et de longueur. `UserService.register` refuse un courriel déjà utilisé, normalise le courriel en minuscules et sans espaces périphériques, encode le mot de passe avec BCrypt, crée un utilisateur `ROLE_USER` et `ACTIVE`, puis crée son profil civil. L'ensemble s'exécute dans une transaction. La réponse `201 Created` est un `UserResponse`, et non l'entité contenant le hash du mot de passe.

Intérêt : conserver les informations d'authentification distinctes des données personnelles, ne jamais stocker le mot de passe en clair et retourner un contrat stable au client. Le statut initial réellement utilisé par le code est `ACTIVE`, même si `PENDING_KYC` existe dans le modèle.

3.2 Connexion et jeton

`POST /users/login` vérifie le DTO, cherche l'utilisateur par courriel et compare le mot de passe reçu au hash BCrypt. Les erreurs « utilisateur absent » et « mauvais mot de passe » utilisent le même message `InvalidCredentialsException`. En cas de succès, `JwtService` signe un JWT RS256 d'une durée de 15 minutes; la réponse fournit `accessToken`, `tokenType=Bearer` et `expiresIn` en secondes.

Intérêt : le client transmet ensuite le jeton dans `Authorization: Bearer <jeton>`, sans que le serveur maintienne une session HTTP. Une clé privée signe les jetons; la clé publique correspondante vérifie les signatures.

3.3 Requête authentifiée

`JwtAuthFilter` récupère le sujet (le courriel) du jeton, retrouve l'utilisateur en base et vérifie à nouveau la validité/expiration du jeton. Il place alors un objet d'authentification contenant cet utilisateur et son autorité dans le contexte Spring Security. `@AuthenticationPrincipal User` permet ensuite aux contrôleurs de recevoir l'utilisateur déjà authentifié.

Le chargement en base signifie que l'état et le rôle actuels de l'utilisateur sont lus à chaque requête, mais implique également une requête de base de données pour chaque jeton traité.

3.4 Dépôt et examen KYC

Un utilisateur authentifié envoie un multipart à `POST /kyc/upload` avec `documentType`, `documentNumber` et un fichier. `KycService.submitDocument` refuse un fichier absent/vide et n'accepte que les extensions PDF/JPG/JPEG/PNG, crée `uploads/kyc` si nécessaire, enregistre le fichier sous un nom UUID, puis crée un `KycRecord` en statut `PENDING`.

L'utilisateur peut consulter ses propres pièces via `GET /kyc/documents`. Un administrateur liste les demandes en attente via `GET /kyc/admin/pending`, puis valide avec `PUT /kyc/admin/documents/{id}/verify` ou rejette avec `PUT /kyc/admin/documents/{id}/reject`. La validation marque le document `VERIFIED`, renseigne `verifiedAt`, efface un éventuel motif précédent et passe l'utilisateur à `ACTIVE`. Le rejet marque le document `REJECTED` et enregistre le motif.

Intérêt : séparer la collecte automatique des pièces de la décision de conformité prise par un administrateur; garder le statut et le motif pour rendre la décision visible. Le stockage des fichiers est local et le code ne fournit pas de route de téléchargement ou de prévisualisation.

4. API HTTP

Méthode et route	Accès configuré	Entrée	Résultat nominal
`POST /users/register`	Public	JSON `RegisterRequest`	`201`, `UserResponse`
`POST /users/login`	Public	JSON `LoginRequest`	`200`, `AuthResponse`
`GET /users/me`	Authentifié	Jeton Bearer	`200`, `UserResponse`
`POST /kyc/upload`	Authentifié	Multipart : type, numéro, fichier	`201`, `KycDocumentResponse`
`GET /kyc/documents`	Authentifié	Jeton Bearer	`200`, liste des pièces du compte courant
`GET /kyc/admin/pending`	`ROLE_ADMIN`	Aucun	`200`, pièces `PENDING`
`PUT /kyc/admin/documents/{id}/verify`	`ROLE_ADMIN`	UUID dans le chemin	`200`, pièce mise à jour
`PUT /kyc/admin/documents/{id}/reject`	`ROLE_ADMIN`	UUID et JSON `KycRejectRequest`	`200`, pièce rejetée
`GET /.well-known/jwks.json`	Public	Aucun	`200`, clé publique JWKS
`/actuator/**`	Public dans la configuration	Selon l'endpoint Actuator exposé	Réponse Actuator

Exemple de connexion (valeurs fictives) :

```
POST /users/login
Content-Type: application/json

{"email":"client@example.test","password":"<mot-de-passe>"}
```

Puis, pour appeler une route protégée :

```
Authorization: Bearer <accessToken>
```

Codes d'erreur prévus : validation invalide `400`, courriel déjà utilisé `409`, identifiants invalides `401`, ressource introuvable `404` lorsqu'une `ResourceNotFoundException` est levée, et erreur inattendue `500`. À noter : la recherche KYC introuvable lève actuellement une `IllegalArgumentException`, non `ResourceNotFoundException`; elle tombe donc dans le gestionnaire générique et retourne `500`.

5. Explication des classes Java

5.1 Démarrage et configuration de sécurité

`IdentityServiceApplication`

Point d'entrée Spring Boot. `main(String[] args)` appelle `SpringApplication.run(...)` pour créer le contexte Spring, découvrir les composants sous le package racine et démarrer le serveur web. `@SpringBootApplication` regroupe la configuration automatique, la configuration Spring et le scan des composants. Le nom de package racine est important : les classes applicatives situées en dessous sont découvertes automatiquement.

`SecurityConfig`

`securityFilterChain(HttpSecurity http)` définit les règles HTTP : désactive CSRF et CORS, choisit les sessions `STATELESS`, autorise les routes d'inscription, de connexion, JWKS et Actuator, réserve `/kyc/admin/**` au rôle admin, puis exige une authentification pour le reste. Le filtre JWT est ajouté avant le filtre d'authentification par nom d'utilisateur/mot de passe.

`passwordEncoder()` expose un `BCryptPasswordEncoder` comme bean Spring. La génération du sel et le calcul du hash sont pris en charge par BCrypt; le service de connexion utilise `matches` au lieu de recalculer/ comparer le hash à la main.

Pourquoi une chaîne de filtres ? Elle centralise les règles d'accès pour toutes les routes au lieu de les répéter dans chaque contrôleur. `hasRole("ADMIN")` attend l'autorité `ROLE_ADMIN`, valeur cohérente avec l'énumération `Role`.

`RsaKeyConfig`

Le constructeur obtient un générateur RSA, le configure en 2048 bits et produit une paire de clés. `rsaPublicKey()` publie la clé publique comme bean; `rsaPrivateKey()` publie la clé privée. Spring injecte ces beans dans le service JWT et le contrôleur JWKS.

Pourquoi RSA ? RS256 sépare la capacité de signature (clé privée) de la capacité de vérification (clé publique), ce qui convient à une architecture où plusieurs services doivent valider un jeton sans pouvoir en émettre.

5.2 Sécurité par jeton

`JwtService`

- Le constructeur reçoit la clé privée et la clé publique.
- `generateToken(User user)` crée les claims `userId`, `role` et `status`; met le courriel dans le sujet; ajoute date d'émission, expiration et identifiant de clé `kid`; signe le jeton avec RS256.
- `getExpirationTime()` convertit la durée configurée dans le code (millisecondes) en secondes pour la réponse HTTP.
- `extractUsername(String token)` lit le sujet.
- `extractClaim(String token, Function<Claims,T> claimsResolver)` factorise l'analyse du payload et l'extraction d'un claim.
- `isTokenValid(String token, String userEmail)` vérifie que le sujet correspond au compte chargé et que l'expiration n'est pas dépassée.
- `isTokenExpired(String token)` compare la date d'expiration avec l'heure actuelle.
- `extractExpiration(String token)` lit le claim d'expiration.
- `extractAllClaims(String token)` analyse les claims signés en vérifiant la signature avec la clé publique.

`jwtExpiration` vaut actuellement 900 000 ms (15 minutes) en dur. Les propriétés `spring.application.security.jwt.*` présentes dans le YAML ne sont pas injectées par cette classe : elles ne pilotent donc pas effectivement la clé ou la durée utilisées par ce code.

`JwtAuthFilter`

- `shouldNotFilter(HttpServletRequest)` exclut du filtre JWT le JWKS, l'inscription, la connexion et les routes Actuator.

- `doFilterInternal(request, response, filterChain)` lit l'en-tête `Authorization`, laisse passer sans authentifier lorsqu'il manque ou ne commence pas par `Bearer `, extrait le jeton, en récupère le sujet, charge l'utilisateur, valide le jeton puis installe `UsernamePasswordAuthenticationToken` avec le rôle. Il poursuit la chaîne dans tous les cas pour que Spring Security applique ensuite les règles.

Si l'extraction échoue (jeton mal formé ou signature incorrecte), le filtre continue sans créer d'authentification; une route protégée est ensuite refusée par la configuration de sécurité. Le code n'envoie pas de réponse de succès sur la seule présence d'un en-tête.

`JwksController`

Le constructeur reçoit la clé RSA publique. `getJwks()` transforme le modulus `n` et l'exposant public `e` en Base64 URL sans padding, puis retourne une structure JSON avec `kty=RSA`, `use=sig`, `alg=RS256`, `kid` et les paramètres de clé. Le `kid` correspond à celui placé dans le JWT afin qu'un vérificateur puisse choisir la bonne clé.

5.3 Contrôleurs REST

`UserController`

- `register(RegisterRequest)` valide le JSON, appelle `UserService.register` et renvoie `201 Created`.
- `login(LoginRequest)` valide le JSON, appelle `AuthService.authenticate` et renvoie `200 OK`.
- `getCurrentUser(User)` reçoit l'utilisateur authentifié, cherche son profil, compose un `UserResponse` avec les champs publics et renvoie `200`. Si le profil est absent, prénom et nom sont `null`.

Le contrôleur injecte `ProfileRepository` directement pour `/me`; cette lecture pourrait aussi être encapsulée dans un service pour uniformiser les responsabilités.

`KycController`

- `uploadDocument(User, DocumentType, String, MultipartFile)` reçoit le multipart et l'identité authentifiée, délègue à `submitDocument`, puis répond `201`.
- `getMyDocuments(User)` ne demande au service que les pièces liées à l'utilisateur authentifié, puis répond `200`.
- `getPendingDocuments()` demande les pièces au statut `PENDING`. La protection admin provient de `SecurityConfig`.
- `verifyDocument(UUID)` délègue la validation de la pièce et répond `200`.
- `rejectDocument(UUID, KycRejectRequest)` valide le motif via `@Valid`, délègue le rejet et répond `200`.

Les contrôleurs sont volontairement minces : ils s'occupent des paramètres HTTP et laissent les règles de traitement aux services.

5.4 Services métier

`UserService`

`register(RegisterRequest)` est transactionnelle. Elle vérifie l'unicité de l'adresse, normalise et encode les informations nécessaires, sauvegarde l'utilisateur avec `saveAndFlush`, associe un `Profile` au compte, sauvegarde ce profil et construit la réponse DTO. L'annotation transactionnelle garantit que les deux sauvegardes sont annulées ensemble si une exception survient.

La vérification préalable du courriel améliore le message fonctionnel; la contrainte `UNIQUE` en base reste la protection finale en cas de deux inscriptions simultanées.

`AuthService`

`authenticate(LoginRequest)` est en lecture seule. Elle recherche l'utilisateur, lève `InvalidCredentialsException` si le compte n'existe pas ou si BCrypt ne reconnaît pas le mot de passe, génère un JWT et construit la réponse d'authentification. Un message d'erreur identique pour les

deux cas évite de révéler lequel des champs est incorrect.

`KycService`

- `submitDocument(User, DocumentType, String, MultipartFile)` contrôle la présence du fichier et son extension, crée le dossier si nécessaire, choisit un nom UUID, copie le contenu, crée le KYC en `PENDING` et renvoie un DTO. Les erreurs d'entrée sont des `IllegalArgumentException`; les problèmes d'I/O sont enveloppés dans une `RuntimeException`.
- `getDocumentsByStatus(KycStatus)` effectue une lecture seule, demande au repository les documents de ce statut et convertit chaque entité en DTO.
- `verifyDocument(UUID)` retrouve la pièce, la marque `VERIFIED`, date la décision, retire le motif de rejet précédent, passe son utilisateur à `ACTIVE`, sauvegarde et retourne le DTO.
- `rejectDocument(UUID, String)` retrouve la pièce, la marque `REJECTED`, enregistre le motif, renseigne `verifiedAt` comme date de décision et retourne le DTO sauvegardé.
- `mapToResponse(KycRecord)` centralise la conversion d'une entité KYC vers `KycDocumentResponse`.
- `getUserDocuments(User)` lit les pièces du propriétaire par identifiant et construit la liste de DTO appartenant à l'utilisateur courant.
- `extractExtension(String)` retourne l'extension minuscule après le dernier point; sans nom ou sans point, retourne une chaîne vide, ensuite refusée par la liste autorisée.

Les méthodes de modification KYC sont transactionnelles; les lectures déclarent `readOnly=true`. Le fichier est cependant un effet de bord du système de fichiers et ne participe pas à la transaction SQL : si la sauvegarde en base échoue après copie, un fichier orphelin peut rester sur disque.

5.5 Persistance : repositories et entités

Repositories

Les repositories étendent `JpaRepository`, qui fournit notamment `save`, `findById`, `findAll` et `delete` sans implémentation manuelle.

- `UserRepository.findByEmail(String)` retourne éventuellement un compte; `existsByEmail(String)` sert à détecter un doublon avant création.
- `ProfileRepository.findByUserId(UUID)` retourne le profil d'un compte.
- `KycRecordRepository.findByUserId(UUID)` liste les justificatifs d'un utilisateur; `findByVerificationStatus(KycStatus)` liste ceux d'un statut donné. Spring Data construit les requêtes à partir des noms de méthodes.

`User`

Entité reliée à la table `users`. Champs : `id` UUID, `email`, `passwordHash`, `role`, `status`, `createdAt`, `updatedAt`. Les rôles et statuts sont stockés sous forme de chaînes (`EnumType.STRING`) afin que les valeurs en base soient lisibles et ne dépendent pas de l'ordre des constantes. `@CreationTimestamp` et `@UpdateTimestamp` gèrent les dates. Lombok fournit getters/setters, constructeurs et builder.

`Profile`

Entité `profiles`, reliée à `User` par une relation un-à-un paresseuse et une colonne `user_id` unique. Elle contient prénom, nom, date de naissance, adresse, téléphone et dates de suivi. Séparer le profil des identifiants permet d'organiser distinctement les attributs de compte et les données civiles.

`KycRecord`

Entité `kyc_records`, liée à `User` par plusieurs-à-un paresseux. Elle stocke type/numéro de document, `fileUrl`, état de vérification, motif, date de décision et dates de suivi. `fileUrl` est ajouté au schéma par la migration V2.

Énumérations

- `Role` : `ROLE_USER`, `ROLE_ADMIN`; sert à l'autorisation par rôle.
- `UserStatus` : `ACTIVE`, `SUSPENDED`, `PENDING_KYC`; représente l'état de compte.
- `KycStatus` : `PENDING`, `VERIFIED`, `REJECTED`; représente la décision sur une pièce.
- `DocumentType` : `PASSPORT`, `NATIONAL_ID`, `DRIVING_LICENSE`; limite les types reconnus aux valeurs prévues.

5.6 DTO et contrats de données

- `RegisterRequest` : courriel, mot de passe, prénom, nom, date de naissance, adresse et téléphone. Les contraintes imposent courriel valide, champs obligatoires et mot de passe d'au moins 8 caractères.
- `LoginRequest` : courriel valide et mot de passe non vide.
- `AuthResponse` : jeton, type et durée de validité.
- `UserResponse` : identifiant, courriel, rôle, statut, prénom, nom et date de création. Le hash du mot de passe n'est jamais inclus.
- `KycRejectRequest` : motif obligatoire et non blanc.
- `KycDocumentResponse` : identifiant de pièce, utilisateur, type et numéro du document, chemin du fichier, état et date de soumission.

Les annotations Lombok (getter, setter, constructeurs, builder) évitent le code répétitif. Les DTO d'entrée utilisent Jakarta Validation; les contrôleurs déclenchent les contrôles avec `@Valid`.

5.7 Exceptions et format d'erreur

- `EmailAlreadyExistsException(String)` porte le conflit d'inscription et le courriel concerné.
- `InvalidCredentialsException()` exprime un échec d'authentification avec un message générique.
- `ResourceNotFoundException(String)` représente une ressource absente et est mappée sur HTTP 404.
- `ErrorResponse` définit les champs JSON communs : statut, erreur, message, chemin, horodatage et erreurs de validation facultatives. `@JsonInclude(NON_NULL)` masque les propriétés absentes.
- `GlobalExceptionHandler.handleValidationExceptions(...)` collecte les erreurs par champ et répond 400.
- `handleEmailAlreadyExists(...)` convertit le doublon en 409.
- `handleResourceNotFound(...)` convertit une ressource introuvable en 404.
- `handleGlobalException(...)` fournit une réponse générique 500 sans renvoyer le détail technique de l'exception.
- `handleInvalidCredentials(...)` répond 401 avec le message métier prévu.

`@RestControllerAdvice` applique ces traitements globalement. Il faut toutefois noter qu'une exception non spécifique, par exemple `IllegalArgumentException` ou certaines erreurs de stockage, peut être convertie en 500 par le gestionnaire générique, même si la cause est une entrée client incorrecte.

6. Base de données et migrations

`V1__init_identity_schema.sql` crée :

- `users`, avec UUID, courriel unique, hash du mot de passe, rôle et statut contraints, dates de création/modification;
- `profiles`, avec un `user_id` unique et une clé étrangère vers `users`; la suppression d'un compte supprime son profil (`ON DELETE CASCADE`);
- `kyc_records`, avec clé étrangère vers `users`, type et statut contrôlés, métadonnées de décision;

- des index sur les colonnes utilisées pour retrouver utilisateurs, profils et pièces KYC.

`V2__add_file_url_to_kyc_records.sql` ajoute `file_url` obligatoire à `kyc_records`, avec une valeur par défaut vide pour permettre la migration des lignes déjà présentes.

Flyway applique les scripts versionnés dans l'ordre et mémorise leur exécution. Hibernate est réglé sur `ddl-auto: validate`, donc il vérifie que les tables correspondent aux entités sans modifier lui-même le schéma. Ce choix évite que l'application et le mécanisme de migration tentent tous deux de changer la base.

Le projet utilise UUID pour les clés principales et les références publiques. Les dates SQL sont des horodatages avec fuseau; le code Java utilise `OffsetDateTime` pour les dates horodatées et `LocalDate` pour la date de naissance.

7. Configuration et lancement

`application.yaml`

- Port HTTP : `8081`.
- Nom Spring : `identity-service`.
- PostgreSQL local par défaut : base `identity_db`, port `5432`; l'URL, l'utilisateur et le mot de passe de connexion peuvent être surchargés par les variables `SPRING_DATASOURCE_*`.
- JPA : validation du schéma et affichage/formatage SQL activés.
- Flyway : actif, migrations sous `classpath:db/migration`.
- Le fichier contient aussi des propriétés JWT sous `spring.application.security.jwt`, mais les classes actuelles ne les lisent pas. La durée de 15 minutes est en dur dans `JwtService`; les clés proviennent de `RsaKeyConfig`.

Démarrage local (Windows, depuis la racine du dépôt) :

```
docker compose up -d postgres-db
.\mvnw.cmd spring-boot:run
```

Démarrage de l'ensemble avec Docker :

```
docker compose up --build
```

Le service écoute sur `http://localhost:8081`; PostgreSQL est publié sur le port 5432. Docker Compose attend le healthcheck de PostgreSQL avant de démarrer le service applicatif et conserve les données de base dans le volume `postgres_data`.

Docker

`Dockerfile` utilise une construction multi-étape : l'étape Maven/Java 21 télécharge les dépendances et produit le JAR, puis une image JRE Java 21 plus légère exécute ce JAR. Le build du conteneur utilise `-DskipTests`; il ne constitue donc pas une preuve que les tests passent. Le dossier `uploads/kyc` est créé dans l'image.

`docker-compose.yml` orchestre PostgreSQL et l'API, configure la connexion interne via le nom de service `postgres-db`, publie les ports et monte `./uploads` pour conserver les fichiers KYC hors du conteneur applicatif. `.dockerignore` exclut notamment les sources Git, les sorties et les fichiers IDE inutiles à l'image. `.gitignore` exclut les artefacts compilés et les fichiers de configuration IDE.

Maven (`pom.xml`)

Le parent Spring Boot fixe une base cohérente de versions. Le projet utilise Java 21 et ajoute les dépendances Spring Web (REST), Security (authentification), Validation, Data JPA, pilote PostgreSQL, Flyway, Actuator, Lombok et JJWT. Les dépendances de tests sont en portée `test`. `spring-boot-maven-plugin` produit l'application exécutable et exclut Lombok du JAR final car Lombok sert à la compilation, pas à l'exécution.

8. Tests

`IdentityServiceApplicationTests.contextLoads()` est un test de fumée : il vérifie que le contexte Spring démarre. Il ne valide pas les parcours d'inscription, de connexion, les autorisations, les transitions KYC, les migrations ni le stockage de fichiers. Le `pom.xml` fournit JUnit, Spring Boot Test, Mockito et Spring Security Test pour étendre la couverture.

Pour exécuter les tests sous Windows :

```
.\mvnw.cmd test
```

Une couverture fonctionnelle utile devrait ajouter des tests de service (mots de passe et transitions KYC), des tests HTTP des contrôleurs (statuts/validations), des tests d'accès (rôle admin/utilisateur), et des tests d'intégration PostgreSQL/Flyway.

9. Points d'attention constatés dans le code

Ces observations décrivent l'implémentation actuelle et ne signifient pas que le périmètre demandé inclut des modifications :

1. **Cycle de vie des clés JWT :** la paire RSA est générée en mémoire à chaque démarrage. Les jetons existants ne seront plus vérifiables après redémarrage et la clé JWKS changera. Un stockage de clés persistant et une stratégie de rotation avec plusieurs `kid` sont nécessaires pour un déploiement durable.
2. **Propriétés JWT non reliées :** le YAML déclare une clé secrète et une durée sous `spring.application.security.jwt`, mais le code utilise RSA et une durée codée en dur. Ces réglages ne pilotent pas le `JwtService`.
3. **Statut à l'inscription :** le nouvel utilisateur est défini `ACTIVE` directement; `PENDING_KYC` est disponible mais n'est pas choisi par `UserService.register`. La validation KYC passe également le compte à `ACTIVE`.
4. **Validation des fichiers :** l'extension est contrôlée, mais le type MIME réel, la taille et le contenu du fichier ne sont pas examinés. L'extension seule ne prouve pas qu'un fichier est un PDF ou une image valide.
5. **Stockage KYC :** les pièces résident dans un dossier local et le chemin est renvoyé dans le DTO. Il n'y a pas de route de consultation/téléchargement dans le code observé. Il faut gérer accès, sauvegardes, rétention, chiffrement et nettoyage des fichiers.
6. **Transaction et fichiers :** une transaction SQL ne peut pas annuler une copie de fichier déjà faite; un échec d'écriture en base peut laisser un fichier sans enregistrement.
7. **Correspondance erreur/HTTP :** `KycService` signale une pièce absente avec `IllegalArgumentException`, alors que le gestionnaire 404 attend `ResourceNotFoundException`; le cas courant devient ainsi une erreur 500.
8. **État du compte à l'authentification :** `AuthService.authenticate` ne refuse pas explicitement un compte `SUSPENDED` ou `PENDING_KYC`; le filtre valide le jeton et retrouve le compte, mais ne bloque pas selon `UserStatus`.
9. **Configuration locale :** les fichiers de configuration de développement contiennent des valeurs par défaut de base de données et des réglages sensibles en dur. Les environnements réels doivent injecter les secrets par un mécanisme dédié et désactiver les réglages de diagnostic non souhaités.
10. **Surface Actuator :** `/actuator/**` est autorisé publiquement dans la chaîne de sécurité. Les endpoints effectivement exposés dépendent de la configuration Spring; il faut choisir explicitement ceux qui sont accessibles dans un environnement exposé.
11. **Autorisation d'accès KYC :** les routes KYC utilisateur reposent sur `@AuthenticationPrincipal` et filtrent par ID dans la méthode de lecture des documents. Les fonctions d'administration sont protégées par rôle dans `SecurityConfig`.

12. **Relations de rôles :** `JwtAuthFilter` construit directement l'autorité à partir de `user.getRole().name()`. Ce format concorde actuellement avec `hasRole("ADMIN")`; toute évolution de format doit préserver cette correspondance.

10. Glossaire

Terme	Sens dans ce projet
API REST	Interface HTTP dont les routes exposent les cas d'utilisation d'identité/KYC
DTO	Objet de transfert définissant les champs envoyés ou reçus par l'API
Entité JPA	Objet Java mappé à une table SQL
Repository	Interface Spring Data qui exécute les opérations de persistance
Transaction	Groupe d'opérations SQL validées ou annulées ensemble
BCrypt	Fonction de hachage de mot de passe avec sel et coût de calcul
JWT	Jeton signé portant des claims, transmis au format Bearer
RS256	Signature JWT RSA avec SHA-256
JWKS	Document JSON décrivant une ou plusieurs clés publiques
KYC	« Know Your Customer » : vérification d'identité par justificatifs
Flyway	Outil qui applique et suit les migrations SQL versionnées
Actuator	Endpoints Spring Boot de santé et de supervision
Lombok	Générateur à la compilation de méthodes Java répétitives

11. Résumé

Le projet fournit une base d'API d'identité structurée en couches avec inscription, authentification JWT RS256, profil, dépôt et examen KYC, persistance PostgreSQL pilotée par Flyway et exécution Docker. Les choix de séparation DTO/entités, de hash BCrypt, de validation centralisée, de migrations SQL et d'autorisation par rôle servent la maintenabilité et la sécurité. Les points cités dans la section précédente délimitent ce qui est réellement implémenté et les sujets à traiter avant un usage bancaire en production.